Expect interview questions that explore function overloading, generics, conditional types, and OOP hierarchies, as well as your ability to design APIs that balance flexibility with type safety. These concepts form the backbone of professional TypeScript development and are critical for excelling in both frontend and full-stack engineering roles.

Table 3.6 Key features of functions, classes, and advanced types

Topic	Fundamentals	Applied skills	Advanced
Function types	Annotate parameters and return types.	Use overloads, optional/default parameters, and rest params effectively.	Model polymorphic and higher-order functions with generics and conditional types.
Classes & interfaces	Define classes, implement interfaces, and apply basic access modifiers.	Use abstract classes, generics in classes, and modifiers (`public`, `private`, `protected`).	Architect type-safe OOP hierarchies with mixins, decorators, and SOLID principles in TypeScript.
Generics	Create generic functions and classes (`<T>`).	Constrain generics with `extends`, use default types, and `infer` with conditional types.	Architect reusable libraries leveraging advanced generic patterns and utility types.
Conditional & mapped types	Recognize basic mapped types (`Partial<T>`, `Readonly<T>`).	Apply mapped types for object transformations; use `keyof`, `in`.	Build complex utility types and solve deep type transformation problems for scalable systems.
Utility types	Use builtin types like `Partial`,	Combine utility types for APIs and	Design custom utility types for